

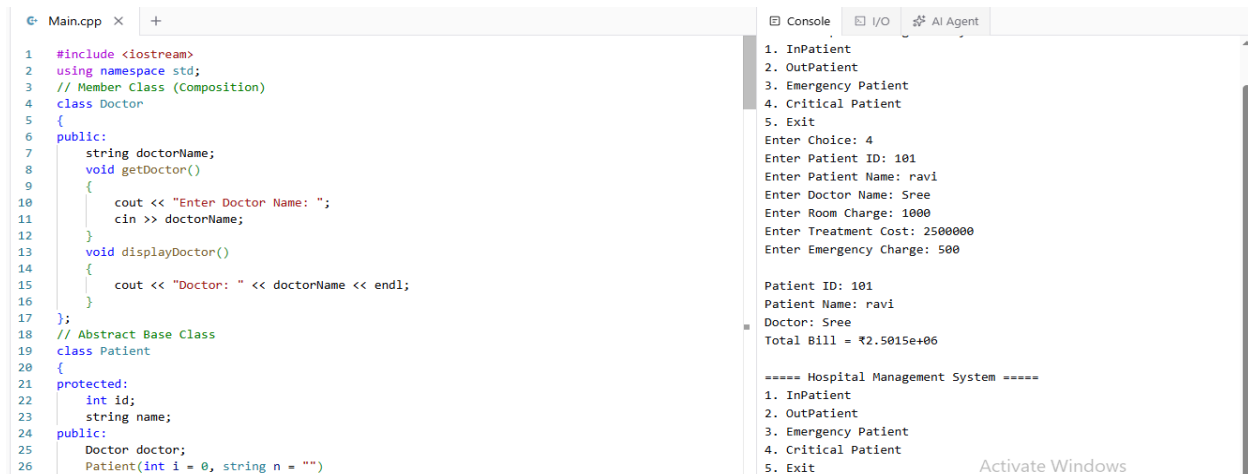
Case Study Task

Name: V. Sree Pavithra Reddy

Regno: 192465020

You are required to analyze, design, implement, and evaluate a C++ application using:

- Inheritance (types of inheritance)
- Virtual base classes
- Abstract classes
- Constructors in derived classes
- Member classes (composition)
- Pointers and pointers to objects
- this pointer
- Pointer to derived classes
- Runtime polymorphism



```
1 #include <iostream>
2 using namespace std;
3 // Member Class (Composition)
4 class Doctor
5 {
6 public:
7     string doctorName;
8     void getDoctor()
9     {
10         cout << "Enter Doctor Name: ";
11         cin >> doctorName;
12     }
13     void displayDoctor()
14     {
15         cout << "Doctor: " << doctorName << endl;
16     }
17 };
18 // Abstract Base Class
19 class Patient
20 {
21 protected:
22     int id;
23     string name;
24 public:
25     Doctor doctor;
26     Patient(int i = 0, string n = "")
```

```
1. InPatient
2. OutPatient
3. Emergency Patient
4. Critical Patient
5. Exit
Enter Choice: 4
Enter Patient ID: 101
Enter Patient Name: ravi
Enter Doctor Name: Sree
Enter Room Charge: 1000
Enter Treatment Cost: 2500000
Enter Emergency Charge: 500

Patient ID: 101
Patient Name: ravi
Doctor: Sree
Total Bill = ₹2.5015e+06

===== Hospital Management System =====
1. InPatient
2. OutPatient
3. Emergency Patient
4. Critical Patient
5. Exit
```

```

14 public:
15 {
16     name = n;
17 }
18 void setDetails(int id, string name)
19 {
20     this->id = id;
21     this->name = name;
22 }
23 virtual void calculateBill() = 0;
24 virtual void display()
25 {
26     cout << "\nPatient ID: " << id << endl;
27     cout << "Patient Name: " << name << endl;
28     doctor.displayDoctor();
29 }
30 virtual ~Patient() {}
31 };
32 // InPatient
33 class InPatient : virtual public Patient
34 {
35 protected:
36     float roomCharge, treatmentCost;
37 public:
38     void input()
39     {
40         cout << "Enter Room Charge: ";
41         cin >> roomCharge;

```

Main.cpp x +

```

50 public:
51 {
52     cout << "Enter Treatment Cost: ";
53     cin >> treatmentCost;
54 }
55 void calculateBill() override
56 {
57     cout << "Total Bill = ₹" << roomCharge + treatmentCost << endl;
58 }
59 };
60 // OutPatient
61 class OutPatient : virtual public Patient
62 {
63 protected:
64     float consultationFee;
65 public:
66     void input()
67     {
68         cout << "Enter Consultation Fee: ";
69         cin >> consultationFee;
70 }
71 void calculateBill() override
72 {
73     cout << "Total Bill = ₹" << consultationFee << endl;
74 }
75 };

```

```

main.cpp
79 // Emergency Patient
80 // Emergency Patient
81 class EmergencyPatient : virtual public Patient
82 {
83 protected:
84     float emergencyCharge;
85 public:
86     void inputEmergency()
87     {
88         cout << "Enter Emergency Charge: ";
89         cin >> emergencyCharge;
90     }
91     void calculateBill() override
92     {
93         cout << "Total Bill = ₹" << emergencyCharge << endl;
94     }
95 };
96 // Multiple Inheritance
97 class CriticalPatient : public InPatient, public EmergencyPatient
98 {
99 public:
100     void calculateBill() override
101     {
102         cout << "Total Bill = ₹"
103         | << roomCharge + treatmentCost + emergencyCharge << endl;
104     }
105 };
106 int main()
107 {

```

```

Main.cpp
.07 {
.10     while (true)
.11     {
.12         cout << "\n==== Hospital Management System =====\n";
.13         cout << "1. InPatient\n";
.14         cout << "2. OutPatient\n";
.15         cout << "3. Emergency Patient\n";
.16         cout << "4. Critical Patient\n";
.17         cout << "5. Exit\n";
.18         cout << "Enter Choice: ";
.19         cin >> choice;
.20
.21         if (choice == 5)
.22         | break;
.23         cout << "Enter Patient ID: ";
.24         cin >> id;
.25         cout << "Enter Patient Name: ";
.26         cin >> name;
.27         Patient *p = nullptr;
.28         switch (choice)
.29         {
.30         case 1:
.31         {
.32             InPatient *ip = new InPatient();
.33             ip->setDetails(id, name);
.34             ip->doctor.getDoctor();
.35             ip->input();
.36             p = ip;

```

```

107 {
111 {
149 {
153     ep->inputEmergency();
154     p = ep;
155     break;
156 }
157 case 4:
158 {
159     CriticalPatient *cp = new CriticalPatient();
160     cp->setDetails(id, name);
161     cp->doctor.getDoctor();
162     cp->InPatient::input();
163     cp->inputEmergency();
164     p = cp;
165     break;
166 }
167 default:
168     cout << "Invalid Choice!" << endl;
169     continue;
170 }
171 p->display();
172 p->calculateBill();
173 delete p;
174 }
175 cout << "Thank You!" << endl;
176 return 0;
177 }

```

Part A – Analysis

1. Limitations

- Duplicate patient data.
- Different billing methods.
- Hard to add new patient types.
- Inefficient memory usage.

2. OOP Solution

- Encapsulation: Protects data.
- Inheritance: Reuses common code.
- Polymorphism: Different billing with one function.
- Abstraction: Hides implementation.

3. Functional Requirements

- Add different patient types.
- Store patient and doctor details.
- Calculate bills.

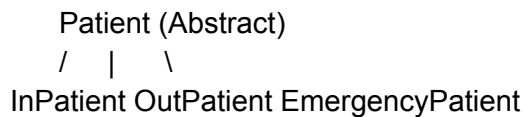
- Display records.
- Support future expansion.

Part B – Design

1. Class Hierarchy

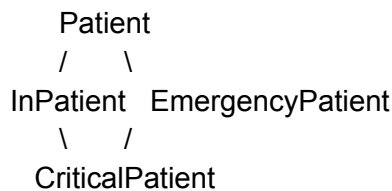
- Abstract Base Class: Patient
- Derived Classes: InPatient, OutPatient, EmergencyPatient

Hierarchy:



2. Multiple Inheritance

Create CriticalPatient by inheriting from both InPatient and EmergencyPatient.



3. Required OOP Concepts

- Virtual Base Class: `virtual public Patient` (avoids duplicate Patient object).
- Member Class (Composition): Doctor class is included inside Patient (Patient has a Doctor)

Part C – Evaluation (Short)

1. Comparison

Inheritance vs Composition

- Inheritance: Is-a relationship.
- Composition: Has-a relationship.

Static vs Dynamic Binding

- Static: Compile-time.
- Dynamic: Run-time (function overriding).

2. Analysis

- Virtual Base Class: Solves the diamond problem.
- Pointers: Enable dynamic memory and runtime polymorphism.

3. Justification

- Design: Uses inheritance, composition, and polymorphism.
- Benefits: Scalable, reusable, and easy to maintain.